# Veri Yapıları

## Algoritma Analizi Temelleri

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

# İçerik-Teorik

- Neden Analiz Ediyoruz? (Performans ve Verimlilik)
- Matematiksel Model: Ne Ölçüyoruz?
- Çalışma Zamanı (Running Time) Kavramı
- Asimptotik Analiz: "Büyük-O" Notasyonu (O)
- Diğer Notasyonlar: Omega (Ω) ve Teta (Θ)
- Kod Analizi İçin Genel Kurallar
- Sık Görülen Büyüme Oranları (Growth Rates)
- Logaritmik (O(log n)) Zaman Kavramı
- Basit Özyinelemeli (Recursive) Fonksiyonların Analizi

# Neden Algoritma Analizi?

- Bir problemi "çözebiliyoruz".
- Peki problemi verimli çözebiliyor muyuz?

# Örnek Problem

- 1 Milyon öğrenci arasında bir öğrencinin numarasını arama
  - Doğrusal arama
  - İkili arama

Örnek Dizi

| 17 | 53 | 61 | 78 | 89 | 92 | 103 | 105 | 116 | 129 | 132 |
|---|---|---|---|---|---|---|---|---|---|---|

1. 105 numaralı öğrenci varmı bulun
2. 61 numaralı öğrenci varmı bulun
3. 132 numaralı öğrenci varmı bulun
4. 17 numaralı öğrenci varmı bulun

# Neden Algoritma Analizi?

- **Neden Sadece Zamanı Ölçmüyoruz?**
- **Bağımlılıklar:**
  - Hangi bilgisayarda çalıştı? (CPU, RAM)
  - Hangi dil/derleyici kullanıldı?
  - Veri seti neydi? (Sadece 10 eleman mı, yoksa 10 Milyar mı?)
- **Sonuç**
  - Bize donanımdan, dilden ve veriden bağımsız, algoritmanın doğasını yansıtan matematiksel bir ölçüt gerekir.

# Matematiksel Model

- **Hedef**
  - Algoritmanın çalışma zamanını, girdi boyutunun (input size) bir fonksiyonu olarak ifade etmek.
- **Girdi Boyutu (n)**
  - **Diziler:** Eleman sayısı.
  - **Matrisler:** Satır x Sütun sayısı.
  - **Graflar:** Düğüm (Vertex) veya Kenar (Edge) sayısı.
- **Ne Sayılır?**
  - En sık yürütülen temel işlem (primitive operation).
  - Genellikle "karşılaştırma" (comparison) işlemi sayılır (örn: if (x < y)).
  - Bazen "atama" (assignment) da sayılabilir.
- **Varsayım**
  - Her temel işlem (atama, karşılaştırma, aritmetik işlem) 1 birim zaman (sabit zaman) alır.

# En İyi (Best), En Kötü (Worst) ve Ortalama (Average) Durum

Örnek

## Doğrusal Arama (Linear Search)

- **En Kötü Durum (Worst Case)**
  - Aradığımız eleman dizinin sonunda veya hiç yok.
  - n adet karşılaştırma gerekir. T(n) = n
- **En İyi Durum (Best Case)**
  - Aradığımız eleman dizinin başında.
  - 1 adet karşılaştırma gerekir. T(n) = 1
- **Ortalama Durum (Average Case)**
  - Elemanın herhangi bir yerde olma olasılığı eşittir.
  - n/2 adet karşılaştırma gerekir. T(n) = n/2

# Hangisi Kritik?

- **En Kötü Durum (Worst Case)**
  - Bir algoritmanın performansının üst sınırını (garantisini) verir. Mühendislik yaklaşımı bunu gerektirir.

# Asimptotik Analiz Nedir?

- $T(n) = 2n^2 + 5n + 10$ gibi bir fonksiyon bulduk diyelim.
- Girdi boyutu (n) çok büyüdüğünde **(sonsuza yaklaşırken)**, bu fonksiyondaki küçük terimlerin ve katsayıların bir önemi kalmaz.
- n=1,000,000 iken ➔
  - $2n^2$ terimi ≈ $2 \times 10^{12}$
  - 5n terimi ≈ $5 \times 10^6$
- $2n^2$ terimi, toplam çalışma zamanını domine eder.
- **Asimptotik Analiz** ➔ Çalışma zamanının büyüme oranına odaklanır.

# Büyük-O Notasyonu (Big-Oh)

- **Tanım:** O(f(n)) - "Büyüme oranı f(n)'den daha hızlı değil." (Üst Sınır - Upper Bound)
- **Sezgisel Anlamı:** En kötü durumda, algoritmamız f(n)'den daha yavaş çalışmayacak.
- **Örnekler**
  - $T(n) = 2n^2 + 5n + 10 \rightarrow O(n^2)$
  - $T(n) = 100n + 50 \rightarrow O(n)$
  - $T(n) = n \times \log(n) + 2n \rightarrow O(n \times \log(n))$
- **Kural:** Düşük dereceli terimleri ve katsayıları at.

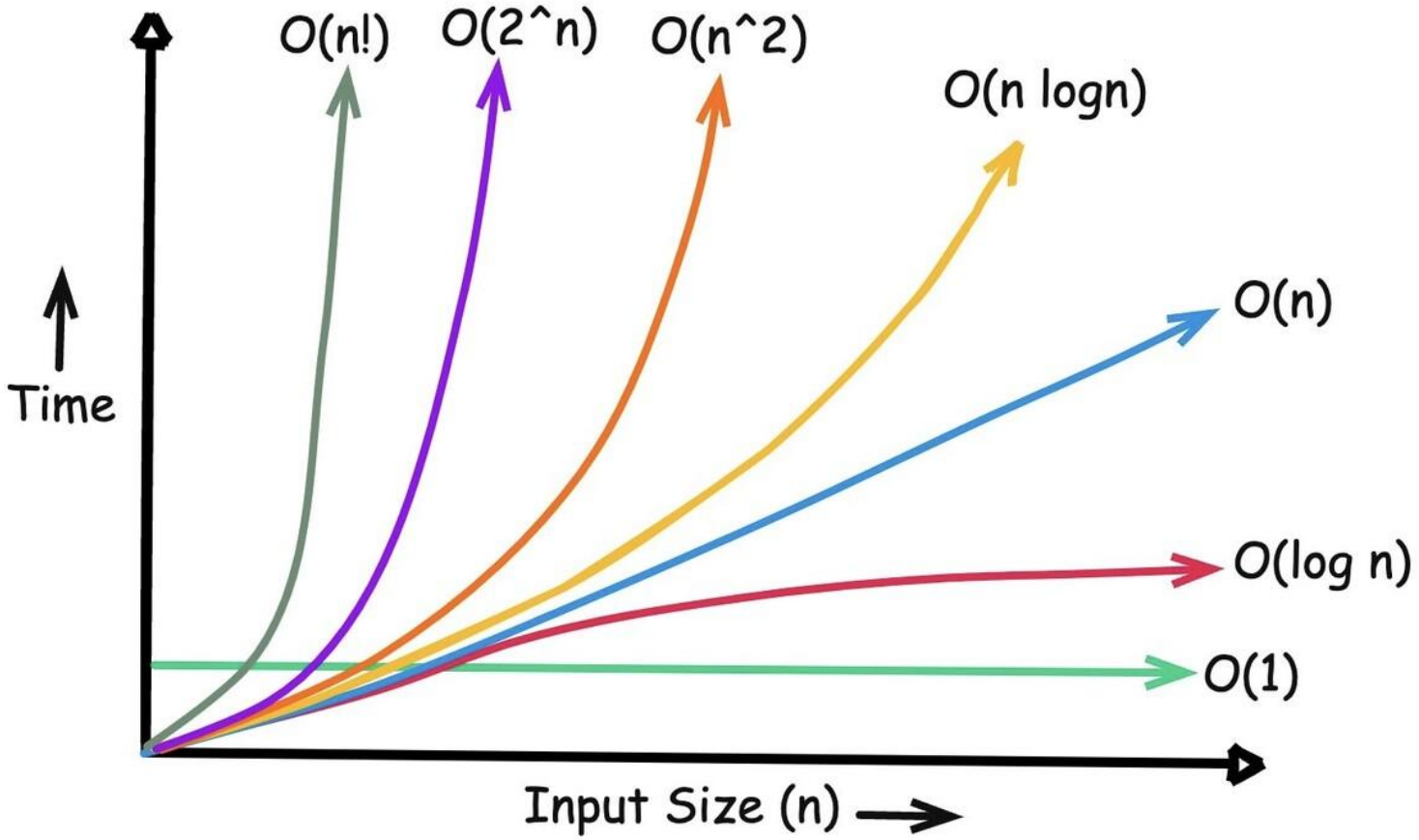
# Omega (Ω) ve Teta (Θ) Notasyonları

- **Büyük-Omega (Ω)**
  - "Büyüme oranı f(n)'den daha yavaş değil." (Alt Sınır - Lower Bound)
  - $T(n) = 2n^2 + 5n \rightarrow \Omega(n^2)$ (Aynı zamanda $\Omega(n)$ de doğrudur, ama "sıkı" değildir).
- **Büyük-Teta (Θ)**
  - "Büyüme oranı tam olarak f(n)." (Sıkı Sınır - Tight Bound)
  - $T(n)$ hem $O(n^2)$ hem de $\Omega(n^2)$ ise, $T(n) \rightarrow \Theta(n^2)$'dir.
- **Pratikte**
  - Genellikle O (Big-Oh) notasyonu, Θ'nin anlamında kullanılır.

# Sık Görülen Büyüme Oranları

- **O(1)** (Constant - Sabit)
  - Girdi boyutundan bağımsız. Örn: Bir dizinin ilk elemanını almak
- **O(log n)** (Logarithmic - Logaritmik)
  - Çok hızlı. Girdi 1 milyar bile olsa ≈ 30 adım sürer. Örn: Binary Search
- **O(n)** (Linear - Doğrusal)
  - Girdi boyutuyla orantılı. Örn: Bir dizideki en büyük elemanı bulmak
- **O(n x log n)** (Log-Linear)
  - Çok verimli. Örn: Verimli sıralama algoritmaları
- **O($n^2$)** (Quadratic - Karesel)
  - Girdi büyüdükçe yavaşlar. Örn: Basit sıralamalar, Bubble Sort
- **O($n^3$)** (Cubic - Kübik)
  - Girdi büyüdükçe yavaşlar. Örn: İki n x n matrisin çarpımı
- **O($2^n$)** (Exponential - Üstel)
  - Kullanılamaz derecede yavaş. n=100 için evrenin yaşından uzun sürer.

# Büyüme Oranlarının Karşılaştırması



# Kod Analizi İçin Genel Kurallar
## Döngüler (Loops)

- Bir for veya while döngüsünün çalışma zamanı, (döngü içindeki işlemlerin zamanı) x (döngünün tekrar sayısı)'dır.
- for (int i=0; i < n; i++) { ... } → O(n) (İçerisi O(1) ise)

# Kod Analizi İçin Genel Kurallar
## İç İçe Döngüler (Nested Loops)

- Çalışma zamanları çarpılır.

```
for (int i=0; i < n; i++) {
    for (int j=0; j < n; j++) { … }
}
```

- → O(n x n) = **$O(n^2)$**

# Kod Analizi İçin Genel Kurallar

## Ardışık İfadeler (Consecutive Statements)

- Çalışma zamanları toplanır ve en büyüğü alınır.
- **Blok 1:** $O(n)$
- **Blok 2:** $O(n^2)$
- **Toplam:** $O(n) + O(n^2) \rightarrow O(n^2)$

# Kod Analizi İçin Genel Kurallar

## Koşul if/else İfadeleri (Condition Statements)

(Koşulun zamanı) + max( if bloğunun zamanı, else bloğunun zamanı)

```
if (number > 0) {
    // code
}
else {
    // code
}
```

# Logaritmik Zaman O(log n) Nasıl Oluşur?

- Bir algoritma, her adımda problem boyutunu sabit bir oranda küçültüyorsa O(log n) zamanlıdır.
  - **Örnek 1:** İkili Arama (Binary Search)
    - Her adımda arama yapılan dizinin yarısı elenir. n → n/2 → n/4 → ... → 1
  - **Örnek 2:** Artan Ritmik Kod Blokları

```
1  for (int i = 1; i < n; i = i * 2) {
2      // ... O(1) işlem
3  }
```

- **i'nin aldığı değerler:** 1, 2, 4, 8, 16, ..., n'e kadar.
- **k adım sonra:** $2^k >= n \Rightarrow k >= \log_2(n)$
- Bu döngü **O(log n)** zamanlıdır.

# Özyineleme (Recursion) Analizi

- Özyinelemeli fonksiyonların analizi "Recurrence Relation" (Yineleme Bağıntısı) ile yapılır.
- **Örnek → Faktöriyel**
  - T(n) = T(n-1) + O(1)
  - Bu O(n) zaman alır (çünkü n defa O(1) işlem yapılır).
- **Örnek → Naif Fibonacci**
  - fib(n) = fib(n-1) + fib(n-2)
  - T(n) = T(n-1) + T(n-2) + O(1)
  - Bu $O(2^n)$ zaman alır (Üstel - Çok verimsiz!)

# Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

# Veri Yapıları

## Algoritma Analizi Temelleri
## (Uygulama Dersi)

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

# İçerik-Uygulama

- Teorik Analiz vs. Ampirik (Deneysel) Analiz
- C++ <chrono> Kütüphanesi ile Zaman Ölçümü
- $O(n^2)$ ve $O(n)$ Algoritmaların Karşılaştırılması
- Kod Parçacıkları Üzerinde Big-O Analizi

# Uygulama 1: Kısmi Toplamlar

- **Amaç**
  - O(n) ve $O(n^2)$ arasındaki farkı somut olarak görmek.
- **Soru**
  - Bir A dizisi verildiğinde, S[i] = A[0] + … + A[i] olacak şekilde S dizisini **hesapla**.
  - Çözüm 1 → **Karesel $O(n^2)$ Çözüm**
    - İç içe döngü kullanılır. Dış döngü i (0'dan n'e), iç döngü j (0'dan m'ye) gider ve S[i] için A[j]'leri toplar.
  - Çözüm 2 → **Doğrusal O(n) Çözüm**
    - Tek döngü kullanılır. S[i] = S[i-1] + A[i] formülünden faydalanılır.



# Uygulama 1: Kısmi Toplamlar

- **Talimatlar**
  - Her iki fonksiyonu da implemente edin.
  - Rastgele sayılarla n = 1000, 5000, 10000, 50000, 100000 boyutlarında diziler oluşturun.
  - <chrono> kütüphanesini kullanarak her iki fonksiyonun bu diziler için çalışma sürelerini (milisaniye veya mikrosaniye cinsinden) ölçün.
  - Sonuçları bir tabloya yazın ve n büyüdükçe $O(n^2)$ çözümün nasıl "patladığını" gözlemleyin.

# Uygulama 2: Logaritmik Zaman

- **Amaç**
  - Big-O analizini pekiştirmek ve O(log n) zamanlı bir algoritma implemente etmek.
- Verilen kod bloklarının Big-O karmaşıklığını bulun (n'e bağlı olarak)

1

```
1  int toplam = 0;
2  for (int i = 0; i < n; i++)
3      for (int j = 0; j < i; j++)
4          toplam++;
```

2

```
1  int toplam = 0;
2  for (int i = 0; i < n; i++)
3      toplam++;
4  for (int j = 0; j < n; j++)
5      toplam++;
```

3

```
1  int toplam = 0;
2  for (int i = n; i > 0; i = i / 2)
3      toplam++;
```

# Uygulama 3: İkili Arama (Binary Search)

- **Yapı**
  - Sıralı bir std::vector<int> ve bir arananDeger alan bir fonksiyon yazın.
  - Fonksiyon, arananDeger dizide varsa true, yoksa false döndürsün.
- **Algoritma**
  - Dizinin sol ve sag indislerini tutun.
  - orta = (sol + sag) / 2 hesaplayın.
  - A[orta] == arananDeger ise → Buldunuz.
  - A[orta] < arananDeger ise → sol = orta + 1 (Sağ yarıda arayın).
  - A[orta] > arananDeger ise → sag = orta - 1 (Sol yarıda arayın).
- **Sonuç**
  - Bu algoritmanın çalışma zamanı neden O(log n)'dir?
    - Çünkü her adımda arama uzayı yarıya bölünür.

# Özet

- Algoritmaların verimliliğini ölçmek için matematiksel bir dil (Asimptotik Notasyon) öğrendik.
- Big-O'nun kod analizi için nasıl kullanılacağını gördük.

# Gelecek Hafta

- Soyut Veri Tipleri (ADT'ler) ve Temel Veri Yapıları
  - Listeler (Diziler ve Bağlı Listeler)

# Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN